VDOC-Prompt: Prompt Engineering for Tanglish Voice Commands — PRD

Oct 28, 2026 · @monick

Overview

VDOC-Prompt finds the best prompt and the best LLM for turning spoken English–Tanglish commands into correct document edits, and proves it with numbers.

Real example of the problem:

- User says: "second paragraph-la bold pannu"
- Correct output: `{"action": "bold", "target": {"type": "paragraph", "index": 2}}`
- A weak prompt outputs: `{"action": "delete", "target": "paragraph"}` (wrong action, no index)

The same model gives very different results depending on how it is prompted. This project measures that difference, the same way an ML project compares SVM vs Random Forest vs CNN.

Goal: reach at least 85% correct commands with under 2 seconds per command, then plug the winning prompt into VDOC.

Objectives and success metrics

1. Build a labelled dataset of 150 English–Tanglish voice commands.
2. Compare 6 prompt techniques across 3 LLMs (18 combinations).
3. Tune the best prompt in 3 rounds (v1 → v2 → v3) using error analysis.
4. Integrate the winning prompt + model into VDOC and demo it live.

Metric	What it means (example)	Target
Exact match accuracy	Action AND target both correct. "bold para 2" → bold, paragraph 2	⩾ 85%
Action accuracy	Only the action is right (bold vs delete)	⩾ 92%
Valid JSON rate	Output parses as JSON with no fixing	⩾ 98%

Metric	What it means (example)	Target
Latency	Seconds from text in to JSON out	⩽ 2 s
Clarification rate on unclear commands	"atha delete pannu" → model asks which one	⩾ 70%

Scope

The project starts from text (the speech-to-text step from VDOC is reused) and ends at a correct JSON command.

In scope

- Text command → JSON action (the "brain" of VDOC)
- 6 prompt techniques, 3 LLMs, full comparison
- Prompt tuning with error analysis
- Streamlit test dashboard + VDOC integration demo

Out of scope

- Improving speech-to-text accuracy
- Fine-tuning or training any model (prompting only)
- Languages other than English and Tanglish

Dataset

150 commands, collected from 10–15 friends speaking naturally, each labelled by hand with the correct JSON. Split: 20 for few-shot examples and tuning, 130 held back for final testing (never shown to the prompt).

Category	Count	Example command	Correct label
Pure English	40	"bold the title"	bold · title
Tanglish	60	"last line-a delete pannu"	delete · line −1
Tamil numbers / words	20	"rendavathu para-va italic aakku"	italic · paragraph 2
Unclear	20	"atha delete pannu"	clarify
Rewrite / tone	10	"intha para-va formal-a maathu"	rewrite · paragraph current · formal

Store as `data/commands.csv` with columns `id, command, category, expected_json`.

```
id,command,category,expected_json
1,bold the title,english,"{""action"":""bold"",""target"":
{""type"":""title""}}"
2,last line-a delete pannu,tanglish,"{""action"":""delete"",""target"":
{""type"":""line"",""index"":-1}}"
3,atha delete pannu,unclear,"{""action"":""clarify"",""question"":""Which
text should I delete?""}"
```

Action schema

Every model output must be one JSON object in this shape, so scoring is automatic and VDOC can run it directly.

```
{
  "action": "bold | italic | underline | delete | insert | rewrite | move |
undo | clarify",
  "target": { "type": "title | paragraph | line | word | selection |
document", "index": 2 },
  "value": "text to insert or tone like formal",
  "question": "only when action is clarify"
}
```

Tanglish word	Means	Action
pannu / aakku	do / make	(depends on the verb before it)
delete pannu / eduthudu	remove	delete
maathu	change	rewrite
serkku / add pannu	add	insert
nagarthu	move	move
munnadi irundha maari	back to before	undo
mudhal / rendavathu / kadaisi	first / second / last	index 1 / 2 / −1

Prompt techniques

Six techniques, each stored as its own file in `prompts/`. `{command}` is replaced with the user's command at run time.

P1 — Zero-shot (`p1_zero_shot.txt`)

```
Convert this document editing command into JSON.
Command: {command}
```

P2 — Role prompting (`p2_role.txt`)

```
You are the command engine of VDOC, a voice document editor used by Tamil
Nadu students.
Users speak English, Tamil words in English letters (Tanglish), or a mix.
Convert the command into JSON.
Command: {command}
```

P3 — Structured output (`p3_structured.txt`)

```
You are the command engine of VDOC, a voice document editor.
Users speak English, Tanglish, or a mix.
Return ONLY one JSON object, no other text, in exactly this format:
{"action": "bold|italic|underline|delete|insert|rewrite|move|undo|clarify",
 "target": {"type": "title|paragraph|line|word|selection|document", "index":
<number or null>},
 "value": <string or null>,
 "question": <string or null>}
Rules:
- "last"/"kadaisi" = index -1. "first"/"mudhal" = 1. "rendavathu" = 2.
- If you cannot tell what to edit, use action "clarify" and ask a short
question.
Command: {command}
```

P4 — Few-shot (`p4_few_shot.txt`) = P3 + these 6 worked examples before the command:

```
Command: bold the title
JSON: {"action":"bold","target":
{"type":"title","index":null},"value":null,"question":null}

Command: last line-a delete pannu
JSON: {"action":"delete","target":
{"type":"line","index":-1},"value":null,"question":null}

Command: rendavathu para-va italic aakku
JSON: {"action":"italic","target":
{"type":"paragraph","index":2},"value":null,"question":null}

Command: intha para-va formal-a maathu
JSON: {"action":"rewrite","target":
{"type":"paragraph","index":null},"value":"formal","question":null}

Command: end-la "Thank you" serkku
JSON: {"action":"insert","target":
{"type":"document","index":-1},"value":"Thank you","question":null}

Command: atha delete pannu
JSON: {"action":"clarify","target":null,"value":null,"question":"Which text
should I delete?"}
```

P5 — Chain-of-thought (`p5_cot.txt`) = P4 + this block before the format:

```
Think in 3 steps before answering:
1. ACTION: which verb is it? (pannu/aakku only mean "do", look at the word
before it)
2. TARGET: what part of the document, and which number?
3. CHECK: is the target clear? If not, use clarify.
Write your steps inside <think></think>, then the JSON on the last line.
```

P6 — Self-consistency = run P5 three times (temperature 0.7) and keep the JSON that appears most often. Ties go to the first run.

All others run at temperature 0, so results repeat the same way each time.

Models

Three free options that cover big cloud, fast open-source, and offline. Check each provider's free-tier limits before the full run; exact model versions are an open choice.

Model	Access	Why include it
Gemini Flash	Google AI Studio API key (free tier)	Strong cloud baseline
Llama 3 (8B or 70B)	Groq API (free tier)	Open-source, very fast replies
Small local model (e.g. Llama 3.2 3B or Qwen 2.5)	Ollama on the laptop	Works offline, no data leaves the device

Total calls ≈ 130 test commands × 18 combinations, with P6 costing 3× = about 3,100 calls (130 × 3 models × 8 runs). Add a 1-second pause between calls to stay under rate limits.

Evaluation

Scoring is fully automatic: a Python script compares each model output with the hand-written label.

1. **Parse** — extract the JSON (for P5, take the last line after `</think>`). Parse fails → counted as wrong and as invalid JSON.
2. **Action match** — `output.action == expected.action`.
3. **Exact match** — action, target type and target index all equal. `value` is checked only for insert (exact text) and rewrite (tone word).
4. **Clarify** — for the 20 unclear commands, correct = action is `clarify`. For clear commands, a `clarify` is counted wrong (asking too much is also a failure).
5. **Latency** — time each call; report the median, not the average, so one slow call doesn't skew it.

Every result row is saved to `results/runs.csv`: `model, technique, id, category, output, valid_json, action_ok, exact_ok, latency_s`. All tables and charts are built from this one file.

Prompt tuning loop

Take the best technique + model from the comparison, then improve it in 3 rounds, like hyperparameter tuning in ML. Tune only on the 20 tuning commands; test each version on the 130 held-out ones.

1. **v1** — best prompt from the comparison, unchanged. Record accuracy.
2. **Error analysis** — list every wrong output and group it. Example groups: "maathu read as move", "rendavathu not read as 2", "asked clarify on a clear command".

3. **v2** — fix the biggest group only. Example: add one few-shot line showing maathu → rewrite. Re-test.
4. **v3** — fix the next group. Example: add the rule "if the command says atha / idhu with no visible target, use clarify". Re-test.
5. **Report** — a line chart of accuracy for v1, v2, v3, plus one before/after example per change.

Change one thing per version, so each gain can be traced to a single change.

Architecture and tech stack

One command flows through the system in 5 steps:

1. **Voice** → VDOC's existing speech-to-text gives text: "last line-a delete pannu".
2. **Prompt builder** → fills the chosen template from `prompts/` with the command.
3. **Model client** → sends it to Gemini, Groq or Ollama through one shared function `call_model(model, prompt)`.
4. **Parser + validator** → pulls out the JSON and checks it against the action schema.
5. **Executor** → VDOC applies the edit, or speaks the clarify question back.

Layer	Tool
Language	Python 3.11
Model APIs	`google-generativeai`, `groq`, `ollama` Python packages
Data and results	CSV files + `pandas`
Charts	`matplotlib`
Dashboard	Streamlit
Secrets	`.env` file with API keys (never pushed to GitHub)

```
vdoc-prompt/
├── data/commands.csv          # 150 labelled commands
├── prompts/                   # p1_zero_shot.txt … p5_cot.txt
├── src/
│   ├── models.py              # call_model() for all 3 providers
│   ├── prompts.py             # load + fill templates, self-consistency
│   ├── evaluate.py            # parse, score, save runs.csv
│   └── report.py              # tables + charts from runs.csv
├── results/runs.csv
├── app.py                     # Streamlit: try a command live + view results
├── requirements.txt
└── .env.example
```

Expected results (hypothesis)

We expect few-shot prompting to lift exact-match accuracy by about 20 points over zero-shot, with cloud models ahead of the local one. These are predictions for the review, not measured numbers; the final report replaces them with real results from `runs.csv`.

Model	Zero-shot	Role	Structured	Few-shot	CoT	Self-consistency
Gemini Flash	~65%	~70%	~78%	~85%	~87%	~90%
Llama 3 (Groq)	~60%	~65%	~74%	~82%	~84%	~87%
Local (Ollama)	~45%	~50%	~60%	~70%	~68%	~75%

Why we expect this

- Zero-shot has never seen "pannu" or "maathu", so it guesses the action.
- Few-shot gives the biggest single jump: one example of "delete pannu" teaches the pattern.
- CoT helps on unclear and Tamil-number commands but adds little on plain English, and may hurt the small local model.
- Self-consistency is most accurate but about 3× slower (~3 s), likely too slow for voice.

Expected recommendation: few-shot + structured output on a cloud model, about 85% at about 1 s per command.

Timeline and deliverables

Four weeks from the first review, one deliverable per week.

Week	Work	Deliverable
1	Collect and label 150 commands; set up API keys and `call_model()`	`commands.csv`, working model calls
2	Write P1–P6; run all 18 combinations	`runs.csv`, comparison table
3	Error analysis; tune v1 → v2 → v3	Tuning chart, error examples
4	Plug the winner into VDOC; Streamlit dashboard; report	Live demo, final report and slides

Risks

Risk	Example	Fallback
Free API rate limits	Groq blocks after too many calls per minute	1–2 s pause between calls; save progress and resume
Model returns text around the JSON	"Sure! Here is the JSON: {...}"	Parser extracts the first `{...}` block; still counted as "needed fixing"
Labels inconsistent	Two people label "move para up" differently	One labelling guide; double-check 20 random rows
Local model too slow on the laptop	8 s per command	Use a smaller model (1–3B) or report it as offline-only
Few-shot examples leak into the test	Same command in examples and test set	Keep the 20 tuning commands strictly separate

Build prompt

Paste this into Claude Code, Cursor or any AI coding tool to generate the project. Paste the Prompt techniques and Action schema sections from this doc right after it.

```
Build a Python 3.11 project called vdoc-prompt on Windows (PowerShell
commands only).

GOAL
Compare prompt techniques and LLMs for turning English–Tanglish document-
editing
commands into JSON actions. Example: "last line-a delete pannu" ->
```

```
{"action":"delete","target":
{"type":"line","index":-1},"value":null,"question":null}

FOLDERS
data/commands.csv (id, command, category, expected_json, split)  split =
tune|test
prompts/p1_zero_shot.txt ... p5_cot.txt  (templates with {command}; text
pasted below)
src/models.py   - call_model(model_name, prompt, temperature) for:
                  "gemini" (google-generativeai), "groq-llama3" (groq),
"ollama-local" (ollama)
                  returns (text, latency_seconds). Keys from .env. Retry once
on error.
src/prompts.py  - load_template(name), build(name, command),
                  self_consistency(model, command): run p5 3x at temperature
0.7, majority vote
src/evaluate.py - parse_json(text): take text after </think> if present,
extract first {...}
                  score(output, expected): valid_json, action_ok, exact_ok
(action + target.type
                  + target.index; value only for insert/rewrite)
                  run_all(): every model x technique (p1..p6) on split=test
rows,
                  1 s sleep between calls, append each row to
results/runs.csv as it finishes,
                  skip rows already in runs.csv so a crash can resume
src/report.py   - from runs.csv: accuracy table (model x technique), accuracy
by category,
                  median latency table, bar chart (grouped by model), save
PNGs to results/
app.py          - Streamlit, 2 tabs:
                  1) "Try it": text box + model dropdown + technique dropdown
-> show JSON + time
                  2) "Results": tables and charts from report.py
requirements.txt, .env.example, README.md with setup steps for Windows

RULES
- Temperature 0 for p1-p5.
- Never crash on bad model output: mark valid_json = False and continue.
- Keep each file under 150 lines, simple and commented for a student.
- Include 10 sample rows in commands.csv so it runs immediately.

[Paste the Prompt techniques and Action schema sections here]
```